

SmartScan AI

Intelligent Document Scanner

Digital Image Processing Mini Project - Project Proposal (Task 1)

GitHub Repository	https://github.com/iJainamJain/ScanSmart-Ai
Dataset (Kaggle) 1	https://www.kaggle.com/datasets/suvroo/scanned-images-dataset-for-ocr-and-vlm-finetuning
Dataset (Kaggle) 2	https://www.kaggle.com/datasets/sthabile/noisy-and-rotated-scanned-documents

Team

Name	Roll Number
Jainam Jain	23108B0084
Dhanush Chowke	23108B0080
Vivek Jaiswal	23108B0082

Problem Statement

Photos of physical documents taken with a phone camera are typically tilted, perspective-distorted, unevenly lit, and cluttered with background. SmartScan AI processes such a photo and produces a clean, flat, scanner-quality digital document using classical image-processing techniques, without relying on black-box ML models for the core pipeline.

Product Vision (Future Goal)

Long-term, SmartScan AI aims to grow into a mobile-first document scanner app - capture a document with a phone camera, get an automatic scan with OCR-based text extraction and searchable PDF export, similar in spirit to commercial scanner apps. This is **not** part of the graded

semester scope: the course project builds and demonstrates the underlying DIP pipeline in Python, which would become the processing engine behind that future app. Keeping the two separate lets the team focus semester effort on the DIP techniques actually being evaluated (filtering, segmentation, morphology, transforms, compression) rather than mobile app development.

Objectives

- Automatically detect a document's boundary in a photo and correct its perspective.
- Enhance the flattened document (contrast, brightness, sharpness) so it is as readable as a real scan.
- Produce a clean, thresholded, scanner-like output ready for export.
- Demonstrate, in an inspectable and explainable way, the core DIP concepts taught this semester.

Pipeline

```
Input Image -> Resizing -> Preprocessing -> Grayscale -> Noise Reduction
-> Edge Detection -> Contour Detection -> Document Boundary Detection
-> Perspective Transform -> Cropping/Flattening -> Enhancement
-> Thresholding/Binarization -> Morphological Processing
-> Final Scanner-Like Output -> Export (Image/PDF)
```

DIP Concept Mapping

Concept area	Techniques used
Image enhancement	Histogram analysis, histogram equalization, CLAHE, contrast/brightness adjustment, sharpening
Filtering	Gaussian blur, median filtering, bilateral filtering
Segmentation	Document/background separation, global & adaptive thresholding
Edge detection	Canny edge detection
Morphological operations	Erosion, dilation, opening, closing
Geometric transforms	Perspective transform, affine transforms, rotation
Image compression	JPEG vs. PNG comparison, PDF size optimization

Development Roadmap

Phase	Focus
1	Repository setup, dataset setup, image loading, basic preprocessing

Phase	Focus
2	Grayscale, blur/filtering, Canny edges, contours, document boundary detection
3	Corner detection, point ordering, perspective transform, flattening
4	Histogram processing, contrast/CLAHE, sharpening, brightness correction
5	Global/adaptive/Otsu thresholding, erosion, dilation, opening, closing
6	JPEG/PNG comparison, compression, PDF generation, multi-page PDF
7	Desktop/web UI (e.g. Streamlit) demo, image upload, preview, export
8 (optional)	OCR, searchable PDF, document management, additional filters

Phases 1-3 are already implemented as a working MVP in the GitHub repository (see link on the cover page).

Evaluation Metrics

- Document detection success rate across the dataset
- Perspective correction quality (visual + geometric)
- Processing time per image
- Output image dimensions
- File size before/after compression
- OCR accuracy (once OCR is added, future phase)
- Visual before/after readability comparison

Tech Stack

Python, OpenCV, NumPy, Matplotlib, Pillow, scikit-image. Future: PyTesseract, ReportLab/FPDF, a GUI framework (Streamlit/Tkinter/PySide).

Dataset

The project uses two complementary sources: two large public Kaggle datasets for the enhancement/segmentation/morphology/compression stages, plus a self-captured dataset for the document-boundary-detection and perspective-correction stage, which needs real photos of tilted documents on cluttered backgrounds - a case the public sets don't cover well.

1. Kaggle - Scanned Images Dataset for OCR and VLM finetuning

Link: <https://www.kaggle.com/datasets/suvroo/scanned-images-dataset-for-ocr-and-vlm-finetuning>
 Size: 3,492 images, 1.82 GB | License: MIT (freely reusable). 10 real-world document categories - advertisements, emails, forms, letters, memos, news, notes, reports, resumes, scientific papers. Used for CLAHE/histogram enhancement, thresholding, morphological operations, and compression comparison.

2. Kaggle - Noisy and Rotated Scanned Documents

Link: <https://www.kaggle.com/datasets/sthabile/noisy-and-rotated-scanned-documents>

Size: 600 scanned images (500 with ground-truth rotation angle labels) | License: data files (c) original authors, attribution required. Captured at non-vertical angles with speckle noise. Used for rotation/skew correction and denoising, with quantitative angle-accuracy evaluation against the provided labels.

3. Self-captured dataset (own images)

Public datasets above are flat, pre-scanned pages - they lack the tilted-photo-on-a-cluttered-background conditions that document boundary detection and perspective correction are actually built to handle. To cover that, the team is capturing its own set of ~20-30 phone photos of documents (receipts, notes, printed pages, assignments, bills, certificates, book pages) on varied surfaces, with deliberate tilt, perspective angle, shadow, and lighting variation. Stored in `dataset/raw/` in the GitHub repository, owned by the project team and used for educational purposes only. This is the primary set used for the core detection -> perspective-transform -> flattening pipeline and for lab-evaluation demos.